```
// === ДОБАВЛЯЕМ ОБЯЗАТЕЛЬНУЮ ВЕРШИННУЮ ТОЧКУ ВХОДА ДЛЯ DIRECTX 11 ===
struct VS_OUTPUT
{
   float4 Pos : SV_POSITION;
   float2 Tex : TEXCOORD0;
};

VS_OUTPUT VS(float4 Pos : POSITION, float2 Tex : TEXCOORD)
{
   VS_OUTPUT output = (VS_OUTPUT)0;
   output.Pos = Pos;
   output.Tex = Tex;
   return output;
}

// ========================================================================
// АППАРАТНЫЕ ТЕКСТУРНЫЕ РЕГИСТРЫ ДЕФЕКТОСКОПА (СТАНДАРТ 2026)
// ========================================================================
Texture2D srcTexture      : register(t0); // Исходный кадр (оригинал 1:1)
Texture2D macroTexture    : register(t1); // Сжатая матрица 4x4 (Режим 3)
Texture2D stretchedTexture : register(t2); // Растянутое макрополе (Режим 4)

// Новые регистры после гамма-коррекции и финала:
Texture2D macroGammaTex   : register(t3); // Сжатая матрица после гамма-коррекции
Texture2D stretchGammaTex  : register(t4); // Растянутое поле после гамма-коррекции
Texture2D resultFilterTex  : register(t5); // Финальный результат работы алгоритма (Режим 5)

SamplerState samplerState0 : register(s0); // Линейный сэмплер видеокарты


// Буфер констант (бывший register(c0))
cbuffer cbData : register(b0)
```

```
{
    float width;     // p0[0]
    float height;    // p0[1]
    float d_width;   // p0[2]
    float d_height;  // p0[3]
    float splitX;    // Положение шторки
    float padding;
};

static const float3 WEIGHT = float3(0.299f, 0.587f, 0.114f);

// Вспомогательная функция чтения и перевода пикселя в монохром (tex2D)
float GetGray(float2 uv)
{
    return dot(srcTexture.Sample(samplerState0, uv).rgb, WEIGHT);
}

// Новая функция: читает пиксель напрямую по его матричному индексу
float GetMacroGray(int2 pos)
{
    // Зажимаем индексы от 0 до максимального размера сжатого буфера (width/8 - 1)
    // чтобы видеокарта никогда не выходила за границы памяти
    int max_x = (int)(width / 8.0f) - 1;
    int max_y = (int)(height / 8.0f) - 1;

    int cl_x = clamp(pos.x, 0, max_x);
    int cl_y = clamp(pos.y, 0, max_y);

    // .Load берет точную ячейку памяти. Третий параметр (0) - это мип-уровень.
    return macroTexture.Load(int3(cl_x, cl_y, 0)).r;
}
```

```
// ========================================================================
// ИСТИННЫЙ ОДНОПРОХОДНЫЙ ДЕФЕКТОСКОП АНАТОЛИЯ ФЕДОРОВИЧА (ОБРАЗЕЦ 2011 ГОДА)
// ========================================================================

// 1. ШЕЙДЕР ДЛЯ ЛЕВОГО ВЕРХНЕГО УЧАСТКА БЛОКА
float4 func_LT(float2 tex_in, float4 screen_pos)
{
   float2 d_xy = float2(d_width / 2.0f, d_height / 2.0f);
   float2 tex_in_3 = tex_in;

   float2 tex_LR = tex_in + float2(d_width * 4.0f, 0.0f);
   float2 tex_TB = tex_in + float2(0.0f, d_height * 4.0f);
   float2 tex_LRTB = tex_in + float2(d_width * 4.0f, d_height * 4.0f);

   float k_S = GetGray(tex_in_3);
   float k_LR = GetGray(tex_LR);
   float k_TB = GetGray(tex_TB);
   float k_LRTB = GetGray(tex_LRTB);

   int i_x = (int(round((tex_in.x + 4.0f * d_xy.x) * width)) % 4) * 2;
   int i_y = (int(round((tex_in.y + 4.0f * d_xy.y) * height)) % 4) * 2;

   float k_SS = ((k_S * (9 + i_x) + k_LR * (7 - i_x)) * (9 + i_y) +
            (k_TB * (9 + i_x) + k_LRTB * (7 - i_x)) * (7 - i_y)) / 256.0f;

   float k1 = GetGray(tex_in);
   float delta = k1 - k_SS;
   float g_Gamma = 1.0f;
   float k_SS_gamma = pow(saturate(k_SS), g_Gamma);

   float out_gray = saturate(k_SS_gamma + delta);
   return float4(out_gray, out_gray, out_gray, 1.0f);
```

```
}

// 2. ШЕЙДЕР ДЛЯ ПРАВОГО ВЕРХНЕГО УЧАСТКА БЛОКА
float4 func_RT(float2 tex_in, float4 screen_pos)
{
   float2 d_xy = float2(d_width / 2.0f, d_height / 2.0f);
   float2 tex_in_3 = tex_in;

   float2 tex_LR = tex_in + float2(d_width * 4.0f, 0.0f);
   float2 tex_TB = tex_in + float2(0.0f, d_height * 4.0f);
   float2 tex_LRTB = tex_in + float2(d_width * 4.0f, d_height * 4.0f);

   float k_S = GetGray(tex_in_3);
   float k_LR = GetGray(tex_LR);
   float k_TB = GetGray(tex_TB);
   float k_LRTB = GetGray(tex_LRTB);

   int i_x = (int(round((tex_in.x + 4.0f * d_xy.x) * width)) % 4) * 2;
   int i_y = (int(round((tex_in.y + 4.0f * d_xy.y) * height)) % 4) * 2;

   float k_SS = ((k_S * (15 - i_x * 2) + k_LR * (1 + i_x * 2)) * (9 + i_y * 2) +
            (k_TB * (15 - i_x * 2) + k_LRTB * (1 + i_x * 2)) * (7 - i_y * 2)) / 256.0f;

   float k1 = GetGray(tex_in);
   float delta = k1 - k_SS;
   float g_Gamma = 1.0f;
   float k_SS_gamma = pow(saturate(k_SS), g_Gamma);

   float out_gray = saturate(k_SS_gamma + delta);
   return float4(out_gray, out_gray, out_gray, 1.0f);
}
```

```
// 3. ШЕЙДЕР ДЛЯ ПРАВОГО НИЖНЕГО УЧАСТКА БЛОКА
float4 func_RB(float2 tex_in, float4 screen_pos)
{
   float2 d_xy = float2(d_width / 2.0f, d_height / 2.0f);
   float2 tex_in_3 = tex_in;

   float2 tex_LR = tex_in + float2(d_width * 4.0f, 0.0f);
   float2 tex_TB = tex_in + float2(0.0f, d_height * 4.0f);
   float2 tex_LRTB = tex_in + float2(d_width * 4.0f, d_height * 4.0f);

   float k_S = GetGray(tex_in_3);
   float k_LR = GetGray(tex_LR);
   float k_TB = GetGray(tex_TB);
   float k_LRTB = GetGray(tex_LRTB);

   int i_x = (int(round((tex_in.x + 4.0f * d_xy.x) * width)) % 4) * 2;
   int i_y = (int(round((tex_in.y + 4.0f * d_xy.y) * height)) % 4) * 2;

   float k_SS = ((k_S * (15 - i_x * 2) + k_LR * (1 + i_x * 2)) * (15 - i_y * 2) +
            (k_TB * (15 - i_x * 2) + k_LRTB * (1 + i_x * 2)) * (1 + i_y * 2)) / 256.0f;

   float k1 = GetGray(tex_in); // Исправлен тип с float2 на float
   float delta = k1 - k_SS;
   float g_Gamma = 1.0f;
   float k_SS_gamma = pow(saturate(k_SS), g_Gamma);

   float out_gray = saturate(k_SS_gamma + delta);
   return float4(out_gray, out_gray, out_gray, 1.0f);
}

// 4. ШЕЙДЕР ДЛЯ ЛЕВОГО НИЖНЕГО УЧАСТКА БЛОКА
float4 func_LB(float2 tex_in, float4 screen_pos)
```

```
{
    float2 d_xy = float2(d_width / 2.0f, d_height / 2.0f);
    float2 tex_in_3 = tex_in;

    float2 tex_LR = tex_in + float2(d_width * 4.0f, 0.0f);
    float2 tex_TB = tex_in + float2(0.0f, d_height * 4.0f);
    float2 tex_LRTB = tex_in + float2(d_width * 4.0f, d_height * 4.0f);

    float k_S = GetGray(tex_in_3);
    float k_LR = GetGray(tex_LR);
    float k_TB = GetGray(tex_TB);
    float k_LRTB = GetGray(tex_LRTB);

    int i_x = (int(round((tex_in.x + 4.0f * d_xy.x) * width)) % 4) * 2;
    int i_y = (int(round((tex_in.y + 4.0f * d_xy.y) * height)) % 4) * 2;

    float k_SS = ((k_S * (9 + i_x * 2) + k_LR * (7 - i_x * 2)) * (15 - i_y * 2) +
                (k_TB * (9 + i_x * 2) + k_LRTB * (7 - i_x * 2)) * (1 + i_y * 2)) / 256.0f;

    float k1 = GetGray(tex_in); // Исправлен тип с float2 на float
    float delta = k1 - k_SS;
    float g_Gamma = 1.0f;
    float k_SS_gamma = pow(saturate(k_SS), g_Gamma);

    float out_gray = saturate(k_SS_gamma + delta);
    return float4(out_gray, out_gray, out_gray, 1.0f);
}

// ======================================================================
// ГЛАВНЫЙ ВЫЗЫВАЮЩИЙ ШЕЙДЕР ФИЛЬТРАЦИИ ДЕФЕКТОСКОПА
// ======================================================================
float4 PS_Stage3(VS_OUTPUT input) : SV_Target
```

```
{
    // 1. Читаем чистый пиксель оригинала из регистра t0
    float4 k1_rgba = srcTexture.Sample(samplerState0, input.Tex);
    float k1 = dot(k1_rgba.rgb, WEIGHT);

    // 2. Читаем УЖЕ ГОТОВОЕ, плавно интерполированное макрополе из регистра t4
    // (Никаких round() и % 4 — видеокарта выдает идеальный гладкий градиент)
    float4 k_SS_rgba = stretchGammaTex.Sample(samplerState0, input.Tex);
    float k_SS = k_SS_rgba.r;

    // 3. Считаем дельту дефектоскопа Анатолия Федоровича
    float delta = k1 - k_SS;
    float g_Gamma = 1.0f;
    float k_SS_gamma = pow(saturate(k_SS), g_Gamma);

    // Выдаем итоговый отфильтрованный кадр
    float out_gray = saturate(k_SS_gamma + delta);
    return float4(out_gray, out_gray, out_gray, 1.0f);
}


// ШЕЙДЕР СКВОЗНОГО КОПИРОВАНИЯ ЭКРАНА С УЧЕТОМ ШТОРКИ ДО/ПОСЛЕ
Texture2D statsTexture : register(t1); // Сюда подадим готовый фильтр из Stage3

float4 PS_Final(VS_OUTPUT input) : SV_Target
{
    if (input.Tex.x < splitX)
    {
        // Слева переводим сырой оригинал в честный монохром по вашим весам
        float4 src_color = srcTexture.Sample(samplerState0, input.Tex);
        float src_gray = dot(src_color.rgb, WEIGHT);
        return float4(src_gray, src_gray, src_gray, 1.0f);
```

```
    }

    // Тонкая разделительная черная линия
    if (abs(input.Tex.x - splitX) < 0.0015f) return float4(0.0f, 0.0f, 0.0f, 1.0f);

    // Справа выводим чистый результат дефектоскопа
    return statsTexture.Sample(samplerState0, input.Tex);
}

// Простой шейдер копирования кадра для ЭТАПА II
float4 PS_Copy(VS_OUTPUT input) : SV_Target
{
    return srcTexture.Sample(samplerState0, input.Tex);
}

// ВРЕМЕННАЯ ЗАГЛУШКА ДЛЯ ИНИЦИАЛИЗАЦИИ ОБЪЕКТА В MAIN.CPP
float4 PS(VS_OUTPUT input) : SV_Target
{
    return srcTexture.Sample(samplerState0, input.Tex);
}

// === ОФИЦИАЛЬНЫЕ ЗАГЛУШКИ ДЛЯ ЧИСТОГО СТАРТА ПРОГРАММЫ БЕЗ ОШИБОК ===

float4 PS_Downsample2X(VS_OUTPUT input) : SV_Target
{
    return srcTexture.Sample(samplerState0, input.Tex);
}

// ========================================================================
// АППАРАТНЫЙ РЕЖИМ 4: ЧЕСТНЫЙ ПОПИКСЕЛЬНЫЙ ШАГ ПО СЕТКЕ МОНИТОРА (1:1)
// ========================================================================
// Новые вспомогательные функции интерполяции макрополя для PS_Stage2
```

```
float4 macro_LT(int2 pos, int i_x, int i_y) {
    float k_S = GetMacroGray(pos);
    float k_LR = GetMacroGray(pos + int2(1, 0));
    float k_TB = GetMacroGray(pos + int2(0, 1));
    float k_LRTB = GetMacroGray(pos + int2(1, 1));
    float k_SS = ((k_S * (9 + i_x) + k_LR * (7 - i_x)) * (9 + i_y) +
            (k_TB * (9 + i_x) + k_LRTB * (7 - i_x)) * (7 - i_y)) / 256.0f;
    //return float4(saturate(k_SS), saturate(k_SS), saturate(k_SS), 1.0f);
    return float4(0.0f, 0.0f, 0.0f, 1.0f);
}

float4 macro_RT(int2 pos, int i_x, int i_y) {
    float k_S = GetMacroGray(pos);
    float k_LR = GetMacroGray(pos + int2(1, 0));
    float k_TB = GetMacroGray(pos + int2(0, 1));
    float k_LRTB = GetMacroGray(pos + int2(1, 1));
    float k_SS = ((k_S * (15 - i_x * 2) + k_LR * (1 + i_x * 2)) * (9 + i_y * 2) +
            (k_TB * (15 - i_x * 2) + k_LRTB * (1 + i_x * 2)) * (7 - i_y * 2)) / 256.0f;
    //return float4(saturate(k_SS), saturate(k_SS), saturate(k_SS), 1.0f);
    return float4(1.0f, 1.0f, 1.0f, 1.0f);
}

float4 macro_RB(int2 pos, int i_x, int i_y) {
    float k_S = GetMacroGray(pos);
    float k_LR = GetMacroGray(pos + int2(1, 0));
    float k_TB = GetMacroGray(pos + int2(0, 1));
    float k_LRTB = GetMacroGray(pos + int2(1, 1));
    float k_SS = ((k_S * (15 - i_x * 2) + k_LR * (1 + i_x * 2)) * (15 - i_y * 2) +
            (k_TB * (15 - i_x * 2) + k_LRTB * (1 + i_x * 2)) * (1 + i_y * 2)) / 256.0f;
    //return float4(saturate(k_SS), saturate(k_SS), saturate(k_SS), 1.0f);
    return float4(0.0f, 0.0f, 0.0f, 1.0f);
}
```

```
float4 macro_LB(int2 pos, int i_x, int i_y) {
   float k_S = GetMacroGray(pos);
   float k_LR = GetMacroGray(pos + int2(1, 0));
   float k_TB = GetMacroGray(pos + int2(0, 1));
   float k_LRTB = GetMacroGray(pos + int2(1, 1));
   float k_SS = ((k_S * (9 + i_x * 2) + k_LR * (7 - i_x * 2)) * (15 - i_y * 2) +
            (k_TB * (9 + i_x * 2) + k_LRTB * (7 - i_x * 2)) * (1 + i_y * 2)) / 256.0f;
   //return float4(saturate(k_SS), saturate(k_SS), saturate(k_SS), 1.0f);
   return float4(1.0f, 1.0f, 1.0f, 1.0f);
}

// Финальный исправленный шейдер растяжения макроструктуры
float4 PS_Stage2(VS_OUTPUT input, float4 pixelPos : SV_Position) : SV_Target
{
   float fX = pixelPos.x;
   float fY = pixelPos.y;

   int tex_x = (int)floor((fX + 0.5f) / 4.0f);
   int tex_y = (int)floor((fY + 0.5f) / 4.0f);

   // Локальный шаг внутри макроблока 4x4
   int i_x = (int)floor(fX + 0.5f) % 4;
   int i_y = (int)floor(fY + 0.5f) % 4;

   // Ветвление для выбора правильного стыка интерполяции в зависимости от четверти
   if (i_x < 2)
   {
      if (i_y < 2) return macro_LT(int2(tex_x, tex_y), i_x, i_y);
      else return macro_LB(int2(tex_x, tex_y), i_x, i_y);
   }
   else
```

```
    {
        if (i_y < 2) return macro_RT(int2(tex_x, tex_y), i_x, i_y);
        else return macro_RB(int2(tex_x, tex_y), i_x, i_y);
    }
}

/*
float4 PS_Stage2(VS_OUTPUT input, float4 pixelPos : SV_Position) : SV_Target
{
    // Сдвиг на 0.5 пикселя макротекстуры для точного попадания в центры блоков 4x4,
    // чтобы компенсировать аппаратное смещение растеризатора Direct3D.
    // (Используем d_width * 2.0f, так как шаг макросетки в 4 раза больше исходного)
    float2 uv_offset = float2(d_width * 2.0f, d_height * 2.0f);
    float2 corrected_uv = input.Tex - uv_offset;

    // Читаем макротекстуру t1 (g_pStageDown4SRV) через линейный сэмплер.
    // Видеокарта аппаратно выполнит ту самую формулу весов Анатолия Федоровича,
    // исключая любые ступенчатые скачки яркости и темные нити на стыках!
    float4 macro_color = macroTexture.Sample(samplerState0, corrected_uv);

    // Возвращаем чистый, идеально интерполированный серый фон
    float out_bg = saturate(macro_color.r);
    return float4(out_bg, out_bg, out_bg, 1.0f);
}
*/

float4 PS_PixelShaderStage2(VS_OUTPUT input, float4 pixelPos : SV_Position) : SV_Target
{
    // Вызываем наш настоящий четырехзонный интерполятор макрополя!
    return PS_Stage2(input, pixelPos);
}
```

```
Texture2D    ss0 : register(t0);
SamplerState ssSampler : register(s0);

float4 main(VS_OUTPUT input) : SV_Target
{
   float2 tex0 = input.Tex;
   float3 WEIGHT = float3(0.299f, 0.587f, 0.114f);

   float GreyValue;
   float GrayMid = 0.0f;
   float GrayMin = 1.0f;
   float GrayMax = 0.0f;
   float GraySqr = 0.0f;

   for(int i = 0; i < 4; i++)
   {
      for(int j = 0; j < 4; j++)
      {
         float4 k1 = ss0.Sample(ssSampler, tex0);
         GreyValue = dot(k1.rgb, WEIGHT);

         GrayMid += GreyValue;
         GraySqr += GreyValue * GreyValue;

         if(GreyValue > GrayMax) GrayMax = GreyValue;
         if(GreyValue < GrayMin) GrayMin = GreyValue;

         tex0.x += d_width;
      }
      tex0.x -= d_width * 4.0f;
      tex0.y += d_height;
   }
```

```
    GrayMid /= 16.0f;
    GraySqr /= 16.0f;
    GraySqr -= GrayMid * GrayMid;
    if(GraySqr > 0.0f) GraySqr = sqrt(GraySqr);

    return float4(GrayMid, GrayMin, GrayMax, GraySqr);
}

// === НАСТОЯЩЕЕ АППАРАТНОЕ СЖАТИЕ БЛОКОВ 4x4 В SHADERS.HLSL ===
float4 PS_Downsample4x4(VS_OUTPUT input) : SV_Target
{
    float3 WEIGHT = float3(0.299f, 0.587f, 0.114f);

    float GreyValue;
    float GrayMid = 0.0f;
    float GrayMin = 1.0f;
    float GrayMax = 0.0f;
    float GraySqr = 0.0f;

    // Обходим блок 4x4 напрямую от исходного изображения
    for(int i = 0; i < 4; i++)
    {
        for(int j = 0; j < 4; j++)
        {
            float2 offset = float2(j * d_width, i * d_height);
            float4 k1 = srcTexture.Sample(samplerState0, input.Tex + offset);

            GreyValue = dot(k1.rgb, WEIGHT);

            GrayMid += GreyValue;
            GraySqr += GreyValue * GreyValue;
```

```
            if(GreyValue > GrayMax) GrayMax = GreyValue;
            if(GreyValue < GrayMin) GrayMin = GreyValue;
        }
    }

    GrayMid /= 16.0f;
    GraySqr /= 16.0f;
    GraySqr -= GrayMid * GrayMid;
    if(GraySqr > 0.0f) GraySqr = sqrt(GraySqr);

    // Возвращаем упакованную макроструктуру блока 4x4
    return float4(GrayMid, GrayMin, GrayMax, GraySqr);
}
```